INSTITUTE OF AERONAUTICAL ENGINEERING
(Autonomous)
Dundigal - 500 043, Hyderabad, Telangana

DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

DATABASE MANAGEMENT SYSTEMS LABORATORY

Name of the Staff :Dr.B.Venkateswara Rao **Emp Id:11207**

EXERCISE –3: QUERIES USING AGGREGATE FUNCTIONS **Date:06/08/2026**

Aim:
To study and implement SQL Aggregate Functions for performing calculations on data stored in database tables.

Aggregate Functions in SQL

Aggregate functions perform calculations on a group of rows and return a single value.

Function	Description	Example
COUNT()	Counts the number of rows	SELECT COUNT(*) FROM EMP;
SUM()	Calculates the total of numeric values	SELECT SUM(SAL) FROM EMP;
AVG()	Calculates the average of numeric values	SELECT AVG(SAL) FROM EMP;
MAX()	Returns the highest value	SELECT MAX(SAL) FROM EMP;
MIN()	Returns the lowest value	SELECT MIN(SAL) FROM EMP;

Below are the SQL queries using the standard Oracle **EMP** and **DEPT** tables.

Create DEPT Table

```
CREATE TABLE DEPT (
   DEPTNO NUMBER(2) PRIMARY KEY,
   DNAME VARCHAR2(20),
   LOC VARCHAR2(20)
);
```

Insert Data into DEPT

```
INSERT INTO DEPT VALUES (10,'ACCOUNTING','NEW YORK');

INSERT INTO DEPT VALUES (20,'RESEARCH','DALLAS');

INSERT INTO DEPT VALUES (30,'SALES','CHICAGO');

INSERT INTO DEPT VALUES (40,'OPERATIONS','BOSTON');
```

Create EMP Table

```
CREATE TABLE EMP (
   EMPNO NUMBER(4) PRIMARY KEY,
   ENAME VARCHAR2(20),
   JOB VARCHAR2(20),
   MGR NUMBER(4),
   HIREDATE DATE,
   SAL NUMBER(8,2),
   COMM NUMBER(8,2),
   DEPTNO NUMBER(2),
   DOB DATE,
   CONSTRAINT FK_DEPT FOREIGN KEY (DEPTNO)
   REFERENCES DEPT(DEPTNO)
);
```

Insert Sample Data

```
INSERT INTO EMP VALUES (7369,'SMITH','CLERK',7902,DATE '2022-01-10',800,NULL,20,DATE '2002-05-15');

INSERT INTO EMP VALUES (7499,'ALLEN','SALESMAN',7698,DATE '2021-03-15',1600,300,30,DATE '2001-04-12');

INSERT INTO EMP VALUES (7521,'WARD','SALESMAN',7698,DATE '2021-04-20',1250,500,30,DATE '2000-11-22');

INSERT INTO EMP VALUES (7566,'JONES','MANAGER',7839,DATE '2020-06-12',2975,NULL,20,DATE '1998-08-18');

INSERT INTO EMP VALUES (7654,'MARTIN','SALESMAN',7698,DATE '2021-08-30',1250,1400,30,DATE '2002-07-10');
```

INSERT INTO EMP VALUES (7698,'BLAKE','MANAGER',7839,DATE '2019-09-15',2850,NULL,30,DATE '1997-02-11');

INSERT INTO EMP VALUES (7782,'CLARK','MANAGER',7839,DATE '2020-02-18',2450,NULL,10,DATE '1998-10-25');

INSERT INTO EMP VALUES (7788,'SCOTT','ANALYST',7566,DATE '2022-07-12',3000,NULL,20,DATE '1999-01-17');

INSERT INTO EMP VALUES (7839,'KING','PRESIDENT',NULL,DATE '2018-11-17',5000,NULL,10,DATE '1995-09-09');

INSERT INTO EMP VALUES (7844,'TURNER','SALESMAN',7698,DATE '2021-09-08',1500,0,30,DATE '2001-03-14');

INSERT INTO EMP VALUES (7876,'ADAMS','CLERK',7788,DATE '2023-01-12',1100,NULL,20,DATE '2003-06-01');

INSERT INTO EMP VALUES (7900,'JAMES','CLERK',7698,DATE '2023-03-15',950,NULL,30,DATE '2002-12-20');

INSERT INTO EMP VALUES (7902,'FORD','ANALYST',7566,DATE '2020-12-03',3000,NULL,20,DATE '1998-04-08');

INSERT INTO EMP VALUES (7934,'MILLER','CLERK',7782,DATE '2023-02-10',1300,NULL,10,DATE '2002-09-16');

COMMIT; 1) Select * from emp; 2) Select * from dept; {write expected output here}

QUERIES USING AGGREGATE FUNCTIONS

1(a) Display employees of department 10 along with average salary

```
SELECT DEPTNO,
      AVG(SAL) AS AVG_SAL
FROM EMP
WHERE DEPTNO=10
GROUP BY DEPTNO;
```

1(b) Display lowest paid employee details under each department

```
SELECT *
FROM EMP E
WHERE SAL=
(
SELECT MIN(SAL)
FROM EMP
WHERE DEPTNO=E.DEPTNO
);
```

1(c) Display number of employees working in each department

```
SELECT DEPTNO,
COUNT(*) AS TOTAL_EMPLOYEES
FROM EMP
GROUP BY DEPTNO;
```

1(d) Display employee count with department name

```
SELECT D.DEPTNO,
D.DNAME,
COUNT(E.EMPNO) AS TOTAL_EMPLOYEES
FROM DEPT D
LEFT JOIN EMP E
ON D.DEPTNO=E.DEPTNO
GROUP BY D.DEPTNO,D.DNAME;
```

1(e) Employees whose names start with B or C

```
SELECT *
FROM EMP
WHERE ENAME LIKE 'B%'
OR ENAME LIKE 'C%';
```

1(f) Employee names whose salary is greater than or equal to 5000

```
SELECT ENAME,SAL
FROM EMP
WHERE SAL>=5000;
```

2(a) Average salary for each job

```
SELECT JOB,
AVG(SAL) AS AVG_SALARY
FROM EMP
GROUP BY JOB;
```

2(b) Average salary of each job excluding Manager

```
SELECT JOB,
AVG(SAL) AS AVG_SALARY
FROM EMP
WHERE JOB<>'MANAGER'
GROUP BY JOB;
```

2(c) Average salary for departments employing more than three people

```
SELECT DEPTNO,
AVG(SAL) AS AVG_SALARY
FROM EMP
GROUP BY DEPTNO
HAVING COUNT(*)>3;
```

2(d) Employees earning more than the lowest salary in department 30

```
SELECT *
FROM EMP
WHERE SAL>
(
SELECT MIN(SAL)
FROM EMP
WHERE DEPTNO=30
);
```

2(e) Demonstrate SIGN(n) function

```
SELECT SIGN(100) AS POSITIVE,
SIGN(0) AS ZERO_VALUE,
SIGN(-100) AS NEGATIVE
FROM DUAL;
```

2(f) Number of days between Date of Birth and current date

```
SELECT EMPNO,
ENAME,
DOB,
TRUNC(SYSDATE-DOB) AS DAYS_LIVED
FROM EMP;
```

QUERIES USING BUILT-IN FUNCTIONS & AGGREGATE FUNCTIONS

3(a) Show two substrings as a single string

```
SELECT CONCAT(SUBSTR('DATABASE',1,4),
        SUBSTR('MANAGEMENT',1,4)) AS RESULT
FROM DUAL;
```

Output

```
RESULT
--------
DATABANA
```

3(b) List all employee names, salary and 15% rise in salary

```
SELECT ENAME,
    SAL,
    SAL*0.15 AS SAL_INCREMENT,
    SAL*1.15 AS NEW_SALARY
FROM EMP;
```

3(c) Display the lowest paid employee details under each manager

```
SELECT *
FROM EMP E
WHERE SAL =
(
SELECT MIN(SAL)
FROM EMP
WHERE MGR = E.MGR
);
```

3(d) Display the average monthly salary bill for each department

```
SELECT DEPTNO,
    AVG(SAL) AS AVG_MONTHLY_SALARY
FROM EMP
GROUP BY DEPTNO;
```

3(e) Show the average salary for all departments employing more than two people

```
SELECT DEPTNO,
    AVG(SAL) AS AVG_SALARY
FROM EMP
GROUP BY DEPTNO
HAVING COUNT(*) > 2;
```

3(f) Display the employee IDs who belong to department 05 along with average salary

If using EMPLOYEE table

```
SELECT DEPTNO,
    AVG(SALARY) AS AVG_SALARY
FROM EMPLOYEE
WHERE DEPTNO = 5
GROUP BY DEPTNO;
```

If using EMP table

```
SELECT DEPTNO,
    AVG(SAL) AS AVG_SALARY
FROM EMP
WHERE DEPTNO = 10
```

```
GROUP BY DEPTNO;
```
(Use the department number available in your table.)

AGGREGATE FUNCTION QUERIES

4(a) Count the number of employees in department 20

```
SELECT COUNT(*) AS TOTAL_EMPLOYEES
FROM EMP
WHERE DEPTNO = 20;
```

4(b) Find the minimum salary earned by Clerk

```
SELECT MIN(SAL) AS MINIMUM_SALARY
FROM EMP
WHERE JOB = 'CLERK';
```

4(c) Find minimum, maximum and average salary of all employees

```
SELECT MIN(SAL) AS MIN_SALARY,
    MAX(SAL) AS MAX_SALARY,
    AVG(SAL) AS AVG_SALARY
FROM EMP;
```

4(d) List the minimum and maximum salaries for each job type

```
SELECT JOB,
    MIN(SAL) AS MIN_SALARY,
    MAX(SAL) AS MAX_SALARY
FROM EMP
GROUP BY JOB;
```

4(e) List the employee names in descending order

```
SELECT ENAME
FROM EMP
ORDER BY ENAME DESC;
```

4(f) List the employee IDs and names in ascending order by employee ID

```
SELECT EMPNO,
    ENAME
FROM EMP
ORDER BY EMPNO ASC;
```

5 a. Find the sids, names of sailors who have reserved all boats called “INTERLAKE”.
b. Find the age of youngest sailor who is eligible to vote for each rating level with at least two such sailors.
c. Find the sname , bid and reservation date for each reservation.
d. Find the ages of sailors whose name begin and end with B and has at least 3 characters. e. List in alphabetic order all sailors who have reserved red boat.
f. Find the age of youngest sailor for each rating level.
The above questions are based on the standard **Sailors–Boats–Reserves** database used in DBMS/SQL labs. Below is a complete table structure, sample records, and queries for **(a)–(f)**.

Qn 5.i) Table Structure

We will use three tables:

SAILORS

Column	Data Type	Description
SID	NUMBER	Sailor ID – Primary Key
SNAME	VARCHAR2(20)	Sailor Name
RATING	NUMBER	Sailor Rating
AGE	NUMBER(4,1)	Sailor Age

BOATS

Column	Data Type	Description
BID	NUMBER	Boat ID – Primary Key
BNAME	VARCHAR2(20)	Boat Name
COLOR	VARCHAR2(10)	Boat Color

RESERVES

Column	Data Type	Description
SID	NUMBER	Sailor ID – Foreign Key
BID	NUMBER	Boat ID – Foreign Key
DAY	DATE	Reservation Date

---

ii). Create Tables

```
CREATE TABLE SAILORS
(
   SID NUMBER PRIMARY KEY,
   SNAME VARCHAR2(20),
   RATING NUMBER,
```

```
   AGE NUMBER(4,1)
);
CREATE TABLE BOATS
(
   BID NUMBER PRIMARY KEY,
   BNAME VARCHAR2(20),
   COLOR VARCHAR2(10)
);
CREATE TABLE RESERVES
(
   SID NUMBER,
   BID NUMBER,
   DAY DATE,
   PRIMARY KEY (SID, BID),
   FOREIGN KEY (SID) REFERENCES SAILORS(SID),
   FOREIGN KEY (BID) REFERENCES BOATS(BID)
);
```

---

iii. Sample Records

SAILORS

```
INSERT INTO SAILORS VALUES (1, 'BROWN', 10, 45);
INSERT INTO SAILORS VALUES (2, 'BOB', 8, 35);
INSERT INTO SAILORS VALUES (3, 'BRUCE', 7, 28);
INSERT INTO SAILORS VALUES (4, 'BEN', 10, 25);
INSERT INTO SAILORS VALUES (5, 'SMITH', 8, 30);
INSERT INTO SAILORS VALUES (6, 'JONES', 7, 20);
INSERT INTO SAILORS VALUES (7, 'BILL', 9, 40);
INSERT INTO SAILORS VALUES (8, 'ADAM', 10, 22);
INSERT INTO SAILORS VALUES (9, 'BARRY', 9, 18);
INSERT INTO SAILORS VALUES (10, 'WILLIAM', 8, 50);
```

BOATS

```
INSERT INTO BOATS VALUES (101, 'INTERLAKE', 'RED');
INSERT INTO BOATS VALUES (102, 'INTERLAKE', 'BLUE');
```

```
INSERT INTO BOATS VALUES (103, 'MARINE', 'RED');
INSERT INTO BOATS VALUES (104, 'OCEAN', 'GREEN');
INSERT INTO BOATS VALUES (105, 'SEA KING', 'RED');
```

RESERVES

```
INSERT INTO RESERVES VALUES (1, 101, DATE '2026-08-01');
INSERT INTO RESERVES VALUES (1, 102, DATE '2026-08-02');

INSERT INTO RESERVES VALUES (2, 101, DATE '2026-08-03');
INSERT INTO RESERVES VALUES (2, 103, DATE '2026-08-04');

INSERT INTO RESERVES VALUES (3, 101, DATE '2026-08-05');
INSERT INTO RESERVES VALUES (3, 102, DATE '2026-08-06');

INSERT INTO RESERVES VALUES (4, 103, DATE '2026-08-07');
INSERT INTO RESERVES VALUES (5, 104, DATE '2026-08-08');

INSERT INTO RESERVES VALUES (6, 105, DATE '2026-08-09');
INSERT INTO RESERVES VALUES (7, 101, DATE '2026-08-10');

INSERT INTO RESERVES VALUES (8, 103, DATE '2026-08-11');
INSERT INTO RESERVES VALUES (9, 105, DATE '2026-08-11');
```

---

5. Query (a)

Find the SIDs and names of sailors who have reserved all boats called "INTERLAKE"

There are two boats named INTERLAKE:

- BID 101
- BID 102

We need sailors who reserved **both** boats.

Query

```
SELECT S.SID, S.SNAME
FROM SAILORS S
WHERE NOT EXISTS
(
   SELECT B.BID
   FROM BOATS B
```

```
    WHERE B.BNAME = 'INTERLAKE'
    AND NOT EXISTS
    (
        SELECT R.BID
        FROM RESERVES R
        WHERE R.SID = S.SID
        AND R.BID = B.BID
    )
);
```

Expected Output

SID	SNAME
1	BROWN
3	BRUCE

Simple explanation

The query checks:

"For every boat whose name is INTERLAKE, has this sailor reserved it?"

This is an example of a **division-type query**.

---

5. Query (b)

Find the age of the youngest sailor who is eligible to vote for each rating level with at least two such sailors.

Assume **voting age = 18 years**.

First, consider sailors whose age is **18 or above**.

Then group them by RATING.

```
SELECT RATING, MIN(AGE) AS YOUNGEST_AGE
FROM SAILORS
WHERE AGE >= 18
GROUP BY RATING
HAVING COUNT(*) >= 2;
```

Expected Output

RATING	YOUNGEST_AGE
8	30
9	18

RATING	YOUNGEST_AGE
10	22

Explanation

WHERE AGE >= 18

Selects voting-eligible sailors.

GROUP BY RATING

Creates groups based on rating.

MIN(AGE)

Finds the youngest sailor in each rating.

HAVING COUNT(*) >= 2

Keeps only rating groups containing at least **2 sailors**.

---

5. Query (c)

Find the SNAME, BID and reservation date for each reservation.

We need information from:

- SAILORS
- RESERVES

Query

SELECT S.SNAME, R.BID, R.DAY

FROM SAILORS S

JOIN RESERVES R

ON S.SID = R.SID;

Expected Output

SNAME	BID	DAY
BROWN	101	01-AUG-26
BROWN	102	02-AUG-26
BOB	101	03-AUG-26
BOB	103	04-AUG-26
BRUCE	101	05-AUG-26
BRUCE	102	06-AUG-26
BEN	103	07-AUG-26
SMITH	104	08-AUG-26
JONES	105	09-AUG-26

SNAME	BID	DAY
BILL	101	10-AUG-26
ADAM	103	11-AUG-26
BARRY	105	11-AUG-26

---

5. Query (d)

Find the ages of sailors whose name begins and ends with B and has at least 3 characters.

We need:

- Name starts with B
- Name ends with B
- At least 3 characters

Use:

LIKE 'B%B'

and

LENGTH(SNAME) >= 3

Query

SELECT AGE

FROM SAILORS

WHERE SNAME LIKE 'B%B'

AND LENGTH(SNAME) >= 3;

However, with the sample data above, there may be no exact match because names such as BOB start and end with **B**, so:

Expected Output

AGE
35

Because:

BOB

↑ ↑

B B

and its length is 3.

If you also want the **name and age**, use:

SELECT SNAME, AGE

FROM SAILORS

WHERE SNAME LIKE 'B%B'

AND LENGTH(SNAME) >= 3;

Output:

SNAME	AGE
BOB	35

---

5. Query (e)

List in alphabetical order all sailors who have reserved a red boat.

Here we need all three tables:

SAILORS → RESERVES → BOATS

Query

```
SELECT DISTINCT S.SNAME
FROM SAILORS S
JOIN RESERVES R
ON S.SID = R.SID
JOIN BOATS B
ON R.BID = B.BID
WHERE B.COLOR = 'RED'
ORDER BY S.SNAME;
```

Expected Output

SNAME
BARRY
BEN
BILL
BOB
BROWN
BRUCE
JONES

Why DISTINCT?

A sailor may reserve more than one red boat. We want the sailor's name **only once**.

Why ORDER BY S.SNAME?

It sorts the names alphabetically.

---

5. Query (f)

Find the age of the youngest sailor for each rating level.

We don't need to check voting eligibility here.

Simply use GROUP BY and MIN().

```
SELECT RATING, MIN(AGE) AS YOUNGEST_AGE
FROM SAILORS
GROUP BY RATING
ORDER BY RATING;
```

Expected Output

RATING	YOUNGEST_AGE
7	20
8	30
9	18
10	22

6. a. List the Vendors who have delivered products within 6 months from orderdate.
   b. Display the Vendor details who have supplied both Assembled and Sub parts.
   c. Display the Sub parts by grouping the Vendor type (Local or Non Local).
   d. Display the Vendor details in ascending order.
   e. Display the Sub part which costs more than any of the Assembled parts.
   f. Display the second maximum cost Assembled part.

The above questions can be implemented using **three tables: VENDOR, PART, and SUPPLY**.

6. Vendor–Parts,supply Database

1. Table Structure

VENDOR

Column	Data Type	Description
VID	NUMBER	Vendor ID – Primary Key
VNAME	VARCHAR2(30)	Vendor Name
VTYPE	VARCHAR2(15)	Local / Non Local
CITY	VARCHAR2(20)	Vendor City

```
CREATE TABLE VENDOR
(
   VID NUMBER PRIMARY KEY,
   VNAME VARCHAR2(30),
   VTYPE VARCHAR2(15),
   CITY VARCHAR2(20)
);
```

PART

Column	Data Type	Description
PID	NUMBER	Part ID – Primary Key
PNAME	VARCHAR2(30)	Part Name
PTYPE	VARCHAR2(15)	Assembled / Sub
COST	NUMBER(10,2)	Part Cost

```
CREATE TABLE PART
(
   PID NUMBER PRIMARY KEY,
   PNAME VARCHAR2(30),
   PTYPE VARCHAR2(15),
   COST NUMBER(10,2)
);
```

SUPPLY

This table records which vendor supplied which part and the order/delivery dates.

Column	Data Type	Description
VID	NUMBER	Vendor ID – Foreign Key
PID	NUMBER	Part ID – Foreign Key
ORDERDATE	DATE	Order Date
DELIVERYDATE	DATE	Delivery Date

```
CREATE TABLE SUPPLY
(
   VID NUMBER,
   PID NUMBER,
   ORDERDATE DATE,
   DELIVERYDATE DATE,
   PRIMARY KEY (VID, PID, ORDERDATE),
   FOREIGN KEY (VID) REFERENCES VENDOR(VID),
   FOREIGN KEY (PID) REFERENCES PART(PID)
);
```

---

2. Sample Records

VENDOR Records

INSERT INTO VENDOR VALUES (101, 'ABC SUPPLIERS', 'Local', 'Hyderabad');

INSERT INTO VENDOR VALUES (102, 'GLOBAL PARTS', 'Non Local', 'Chennai');

INSERT INTO VENDOR VALUES (103, 'SRI VENDORS', 'Local', 'Vijayawada');

INSERT INTO VENDOR VALUES (104, 'TECH SOLUTIONS', 'Non Local', 'Bangalore');

INSERT INTO VENDOR VALUES (105, 'AP COMPONENTS', 'Local', 'Visakhapatnam');

Check:

SELECT * FROM VENDOR;

Output:

VID	VNAME	VTYPE	CITY
101	ABC SUPPLIERS	Local	Hyderabad
102	GLOBAL PARTS	Non Local	Chennai
103	SRI VENDORS	Local	Vijayawada
104	TECH SOLUTIONS	Non Local	Bangalore
105	AP COMPONENTS	Local	Visakhapatnam

---

3. PART Records

INSERT INTO PART VALUES (201, 'Motherboard', 'Assembled', 50000);

INSERT INTO PART VALUES (202, 'Keyboard', 'Sub', 1200);

INSERT INTO PART VALUES (203, 'Monitor', 'Assembled', 15000);

INSERT INTO PART VALUES (204, 'RAM', 'Sub', 8000);

INSERT INTO PART VALUES (205, 'Hard Disk', 'Sub', 7000);

INSERT INTO PART VALUES (206, 'CPU Cabinet', 'Assembled', 12000);

INSERT INTO PART VALUES (207, 'SSD', 'Sub', 18000);

INSERT INTO PART VALUES (208, 'Printer', 'Assembled', 10000);

Check:

SELECT * FROM PART;

---

4. SUPPLY Records

INSERT INTO SUPPLY VALUES

(101, 201, DATE '2026-01-10', DATE '2026-02-15');

INSERT INTO SUPPLY VALUES

(101, 202, DATE '2026-02-01', DATE '2026-03-10');

INSERT INTO SUPPLY VALUES

(102, 203, DATE '2026-01-15', DATE '2026-05-10');

INSERT INTO SUPPLY VALUES

(102, 204, DATE '2026-02-10', DATE '2026-04-15');

INSERT INTO SUPPLY VALUES

(103, 205, DATE '2026-03-01', DATE '2026-04-10');

INSERT INTO SUPPLY VALUES

(103, 206, DATE '2026-03-15', DATE '2026-05-20');

INSERT INTO SUPPLY VALUES

(104, 207, DATE '2026-04-01', DATE '2026-05-15');

INSERT INTO SUPPLY VALUES

(104, 208, DATE '2026-04-10', DATE '2026-07-01');

INSERT INTO SUPPLY VALUES

(105, 202, DATE '2026-05-01', DATE '2026-06-01');

INSERT INTO SUPPLY VALUES

(105, 207, DATE '2026-05-10', DATE '2026-08-01');

(a) List the Vendors who have delivered products within 6 months from orderdate

We need to compare:

DELIVERYDATE <= ORDERDATE + 6 months

In Oracle, use ADD_MONTHS().

Query

SELECT DISTINCT V.VID, V.VNAME, V.VTYPE, V.CITY

FROM VENDOR V

JOIN SUPPLY S

ON V.VID = S.VID

```
WHERE S.DELIVERYDATE <= ADD_MONTHS(S.ORDERDATE, 6);
```

Explanation

ADD_MONTHS(S.ORDERDATE, 6)

means:

Add 6 months to the order date.

Then:

DELIVERYDATE <= ADD_MONTHS(ORDERDATE, 6)

means the product was delivered **within 6 months**.

If you want to display order and delivery dates also:

```
SELECT V.VID, V.VNAME,
    S.ORDERDATE,
    S.DELIVERYDATE
FROM VENDOR V
JOIN SUPPLY S
ON V.VID = S.VID
WHERE S.DELIVERYDATE <= ADD_MONTHS(S.ORDERDATE, 6);
```

(b) Display the Vendor details who have supplied both Assembled and Sub parts

We need vendors who supplied:

1. At least one Assembled part
2. At least one Sub part

Query using GROUP BY and HAVING

```
SELECT V.VID, V.VNAME, V.VTYPE, V.CITY
FROM VENDOR V
JOIN SUPPLY S
ON V.VID = S.VID
JOIN PART P
ON S.PID = P.PID
WHERE P.PTYPE IN ('Assembled', 'Sub')
GROUP BY V.VID, V.VNAME, V.VTYPE, V.CITY
HAVING COUNT(DISTINCT P.PTYPE) = 2;
```

Important concept

COUNT(DISTINCT P.PTYPE) = 2

means the vendor must have supplied **both types**.

For example:

Vendor 101

Motherboard → Assembled

Keyboard → Sub

Therefore Vendor 101 qualifies.

(c) Display the Sub parts by grouping the Vendor type (Local or Non Local)

First, select only Sub parts and group them according to vendor type.

Query

```
SELECT V.VTYPE, P.PID, P.PNAME, P.COST
FROM VENDOR V
JOIN SUPPLY S
ON V.VID = S.VID
JOIN PART P
ON S.PID = P.PID
WHERE P.PTYPE = 'Sub'
ORDER BY V.VTYPE;
```

Expected type of output

VTYPE	PID	PNAME	COST
Local	202	Keyboard	1200
Local	205	Hard Disk	7000
Local	207	SSD	18000
Non Local	204	RAM	8000
Non Local	207	SSD	18000

If the question specifically means grouping/counting Sub parts by vendor type, use:

```
SELECT V.VTYPE, COUNT(DISTINCT P.PID) AS SUB_PARTS
FROM VENDOR V
JOIN SUPPLY S
ON V.VID = S.VID
JOIN PART P
ON S.PID = P.PID
WHERE P.PTYPE = 'Sub'
GROUP BY V.VTYPE;
```

Example:

VTYPE	SUB_PARTS
Local	3
Non Local	2

(d) Display the Vendor details in ascending order

Usually this means displaying vendor details in ascending order of **vendor name**.

```
SELECT *
FROM VENDOR
ORDER BY VNAME ASC;
```

Output will be alphabetically ordered:

ABC SUPPLIERS

AP COMPONENTS

GLOBAL PARTS

SRI VENDORS

TECH SOLUTIONS

If they want ascending order based on Vendor ID:

```
SELECT *
FROM VENDOR
ORDER BY VID ASC;
```

(e) Display the Sub part which costs more than any of the Assembled parts

This question is important because of the word **"any"**.

Meaning

Find a **Sub part** whose cost is greater than **every assembled part**.

For our assembled parts:

Part	Type	Cost
Motherboard	Assembled	50000
Monitor	Assembled	15000
CPU Cabinet	Assembled	12000
Printer	Assembled	10000

The maximum assembled cost is:

50000

Therefore, a Sub part must cost **more than 50000**.

Our sample data has no such Sub part.

Using MAX()

The same question can be written more simply:

```
SELECT PID, PNAME, COST
FROM PART
WHERE PTYPE = 'Sub'
AND COST >
(
   SELECT MAX(COST)
   FROM PART
   WHERE PTYPE = 'Assembled'
);
```

This is often easier for students to understand.

(f) Display the second maximum cost Assembled part

We need the **second highest cost** among Assembled parts.

Our assembled parts are:

PNAME	COST
Motherboard	50000
Monitor	15000
CPU Cabinet	12000
Printer	10000

Therefore:

Maximum = 50000

Second Maximum = 15000

Query 1 – Using MAX()

```
SELECT PID, PNAME, COST
FROM PART
WHERE PTYPE = 'Assembled'
AND COST =
(
   SELECT MAX(COST)
   FROM PART
```

```
    WHERE PTYPE = 'Assembled'
    AND COST <
    (
        SELECT MAX(COST)
        FROM PART
        WHERE PTYPE = 'Assembled'
    )
);
```

Output

PID	PNAME	COST
203	Monitor	15000
